TABLE DE SYMBOLES

PARTAGE DE VARIABLE ENTRE FLEX ET BISON

Comment Bison peut-il traiter les valeurs des entités lexicales, sachant que l'analyseur lexical doit coder toutes les entités du programme source ?

Solution :

- Flex peut transmettre la valeur d'une entité avant de la coder en utilisant la variable prédéfinie **yylval**.

-> La variable **yylval** est nécessaire pour détecter les erreurs sémantiques. L'analyseur lexical l'enverra donc en complément du code de l'entité.

La variable ‘yylval’

- **yylval** est une variable globale utilisée par Flex et Bison pour transmettre une **valeur** du lexical vers la syntaxique .
- Elle permet donc au scanner (Flex) d’envoyer non seulement le type du token, mais aussi sa **valeur** exacte au parseur (Bison).

L'utilisation de la yylval

1. Déclaration

Lexical.l

```
//Declaration de yylval dans flex
%{
extern YYSTYPE yylval;
%}
```

Syntax.y

```
//YYSTYPE : C'est le type de yylval
//déclaré avec %union
%union {
int   entier;
char* str;
float numvrg;
}
```

2. Utilisation

2.1. Transmission de la valeur de l'entité depuis le fichier .l

Note:

- atoi() : Convertit une chaîne de caractères représentant un entier en un type entier.
- atof(): Convertit une chaîne de caractères représentant un nombre réel en un type réel.

Lexical.l

```
%%
{cst} { yylval.entier=atoi(yytext);
        return cst; }

{idf} {yylval.str=strdup(yytext);
      return idf; }
{reel} { yylval.numvrg=atof(yytext);
return reel;}
 ....
%%
```

2. Utilisation

2.2. Traitement des valeurs dans le fichier .y

Syntax.y

```
%token <str>idf   <entier>cst <numvrg> reel
%%
s : idf aff  idf  div cst  pvg  { if ($5==0)  {printf(" erreur : division par zéro");}
else printf("la division est%s= %s/%d", $1,$3,$5);
        YYACCEPT;
}
....
.%%
```

Remarques :

- En C, on ne peut pas comparer directement deux chaînes de caractères avec l'opérateur == car cela comparerait les adresses mémoire des chaînes et non leur contenu.
- La fonction **strdup** en C crée une copie d'une chaîne de caractères en allouant dynamiquement de la mémoire pour cette copie.
- Le symbole **$$** référence la valeur associée au **non terminal** de la **partie gauche** d'une règle de grammaire.
- Le symbole **$i** référence la valeur associée au **i ème symbole terminal ou non terminal** de la **partie droite** d'une règle de grammaire.

TABLE DE SYMBOLES

- La table des symboles (TS) est une structure cruciale dans un compilateur.
- Elle permet de stocker et de retrouver toutes les informations relatives aux entités du programme (variables, fonctions, constantes, etc.).
- Elle doit être implémentée manuellement dans Flex et Bison.
- Chaque entité reconnue par le langage doit être insérée dans la table dès son analyse.
- Elle est très utile pour détecter les erreurs sémantiques (types incompatibles, variables non déclarées, doublons, etc.).

Programme source

Program L3_Compil
DEC
integer x,y,z;
Const reel a;
DEBUT
a=1;
y=0;
FIN

State	NomEntité	CodeEntité	Type	Val
1	L3_compil	Idf		
1	x	Idf	integer	
1	y	Idf	integer	0
1	z	Idf	integer	
1	a	Idf	reel	1
	.....			

TABLE DE SYMBOLES
1 ère version

- Dans ce TP, nous distinguons 3 types de TS:
  - La TS des IDFs et des constantes
  - La TS des mots clés
  - La TS des séparateurs

Déclaration de la TS des constantes & variables

Lexical.l

```
%{ typedef struct
{
  int state;
  char name[20];
  char code[20];
  char type[20];
  char val[20];
 } TypeTS;
TypeTS TS[200];
%}
```

Déclaration de la TS des Mots clés & Séparateurs

Lexical.l

```
%{
  typedef struct
  { int   state;
     char nomEntite[20];
     char Code[20];
   } TypeSM;
TypeSM  tabS[50], tabM[50];
%}
```

- **State:** permet de vérifier si la ligne dans la TS est occupée ou non pour stocker une entité
- **Name:** Nom de l'entité
- **Code :** IDF, Mot clé, ...etc
- **Type :** type de l'entité (integer, Float ...etc)

Fonction d'initialisation de la TS

- Elle sert à initialiser les structures de données qui vont stocker les informations relatives aux symboles.
- Elle s'assure qu'aucune case n'est marquée comme occupée avant de commencer à remplir la table de symboles.

```
void initialization()
{
int i;
// TS des constantes & variables
for (i=0;i<200; i++)  TS[i]. state=0;
// TS des  Mots clés & Op
for (i=0;i<50;i++)
  {
    tabS[i].state=0;
    tabM[i].state=0;
  }
}
```

Fonction de recherche dans la TS

- La fonction **recherche:** a pour rôle de vérifier si une entité existe ou non dans la table des symboles.
  - Si l'entité n'existe pas, elle l'insère dans la table.
  - Sinon, elle retourne que l'entité existe déjà.

- Elle prend en Entrée:
  - Entité .
  - Code (Mot clé, Idf , cst, var..etc)
  - Type : Integer, Float.
  - val : valeur de l’entité si elle possède
  - y :pour indiquer dans quelle table je vais insèrer l’entité si elle n’existe pas dans la table.

```
void Rechercher(char entite[], char code[],char type[],char val[],int y)
{ int j,i;
switch(y)
 { case 1:  /*1:  la tables des IDF et CONST */
   for (i=0;((i<200)&&(TS[i].state==1)&&(strcmp(entite,TS[i].name)!=0));i++);
   if(i<200 && strcmp(entite,TS[i].name)!=0)
   {  inserer(entite,code,type,val,i,1);  // i -> position d'insertion
                                          // 1 -> Recherche dans TS des idfs & consts
                                          // 2-> Recherche dans la TS des mots clés
                                          // 3 -> Recherche dans la TS des séparateurs
    }
   else  printf(">>>> L'entite %s existe deja \n",entite);
   break;
```

```
case 2: /*la table des mots clés */

  for (i=0;((i<50)&&(tabM[i].state==1))&&(strcmp(entite,tabM[i].name)!=0);i++);
   if(i<50 && strcmp(entite,tabM[i].name)!=0) inserer(entite,code,type,val,i,2);
  else
   printf(">>>> L'entite %s existe deja \n",entite);
  break;

 case 3:/*la table des séparateurs*/
   for (i=0;((i<50)&&(tabS[i].state==1))&&(strcmp(entite,tabS[i].name)!=0);i++);
  if(i<50 && strcmp(entite,tabS[i].name)!=0)  inserer(entite,code,type,val,i,3);
  else   printf(">>>> L'entite %s existe deja \n",entite);
  break;
} // fin -switch case
} // fin de la fonction
```

Fonction d'insertion dans la TS

- Fonction permettant d'insérer les entités du programme dans la table des symboles si elles n'y existent pas déjà.

```
void inserer (char entite[], char code[],char type[],char val[],int i, int y)
{ switch (y)
 { case 1:/*insertion dans la table des IDF et CONST*/
   TS[i].state=1;
   strcpy(TS[i].name,entite);
   strcpy(TS[i].code,code);
  strcpy(TS[i].type,type);
  strcpy(TS[i].val,val);
  break;
 case 2: /*insertion dans la table des mots clés*/
   tabM[i].state=1;
   strcpy(tabM[i].name,entite);
   strcpy(tabM[i].code,code);
   break;
 case 3:/*insertion dans la table des séparateurs*/
  tabS[i].state=1;
  strcpy(tabS[i].name,entite);
  strcpy(tabS[i].code,code);
  break;
 }
```

Fonction d'affichage de la TS

- Pour afficher le contenu de la table de symboles.

```
void afficher()
{int i;

printf("/**************Table des symboles IDF*************/\n");
printf("_____________________________________________________________________________\n");
printf("\t| Nom_Entite | Code_Entite | Type_Entite | Val_Entite \n");
printf("_____________________________________________________________________________\n");

for(i=0;i<200;i++)
{
 if(TS[i].state==1)
  { printf("\t|%10s |%15s | %12s | %12s \n",TS[i].name,TS[i].code,TS[i].type,TS[i].val);
  }
}
```

Suite ...

```
printf("\n/**************Table des symboles mots clés*************/\n");
printf("_____________________________________\n");
printf("\t| NomEntite | CodeEntite | \n");
printf("_____________________________________\n");
for(i=0;i<50;i++)
 if(tabM[i].state==1)
  { printf("\t|%10s |%12s | \n",tabM[i].name, tabM[i].type);
  }
printf("\n/**************Table des symboles séparateurs*************/\n");
printf("_____________________________________\n");
printf("\t| NomEntite | CodeEntite | \n");
printf("_____________________________________\n");
for(i=0;i<50;i++)
 if(tabS[i].state==1)
  { printf("\t|%10s |%12s | \n",tabS[i].name,tabS[i].type );
  }
}
}
```

Utilisation du fichier TS.h

- La déclaration de la table des symboles (TS) et les fonctions associées peuvent encombrer le fichier lexical.l.
- Pour plus de clarté, il est conseillé de les déplacer dans un fichier d'en-tête, par exemple **TS.h**, et de l'inclure dans **lexical.l** avec: **#include "TS.h"**

TS.h

```
//Structure de la TS
typedef struct
{
 int state;
 char name[20];
 char code[20];
 char type[20];
 char val[20];
} TypeTS;

typedef struct
{
 int state;
 char name[20];
 char type[20];
} TypeSM;

TypeTS TS[200];
TypeSM tabS[50],tabM[50];
```

```
void initialization();
{ ...
}

void inserer(char entite[], char code[],char type[],char val[],int i, int y)
{ ...
}

void recherche(char entite[], char code[],char type[],char val[},int y)
{ ...
}

void afficher()
{ ...
}
```

Exemple:

Voici le programme suivant:

```
#import Array
#import Stdio
Programme ACAD_B
{
  VAR  INTEGER : A // B // X $
  VAR  INTEGER : Tab[5] $
  CONST  INTEGER : C=5// R=15 $
  A:=5$
  B:=5$
  A:=A/0$
  B:=C$
}
```

Lexical.l

```
%{ int nb_ligne=1;
 #include "syn.tab.h"
 #include "TS.h"
%}
lettre [a-zA-Z]
chiffre[0-9]
IDF{lettre}({lettre}|{chiffre}|_)*
cst{chiffre}+
%%
import {rechercher(yytext,"Mot cle","","",2); return mc_import; }
Stdio { rechercher(yytext,"Mot cle","","", 2); return mc_stdio; }
Array  { rechercher(yytext,"Mot cle","","", 2); return mc_array;}
Programme { rechercher(yytext,"Mot cle","","", 2); return mc_pgm; }
INTEGER { rechercher(yytext,"Mot cle","","", 2); return mc_entier; }
VAR  { rechercher(yytext,"Mot cle","","", 2); return mc_var ;}
CONST  { rechercher(yytext,"Mot cle","","", 2); return mc_const; }
{IDF}  { rechercher(yytext,"IDF","","",1); return idf;}
{cst}  { return cst;}
"/"  { rechercher(yytext,"Separateur","","",3); return divis;}
"#"  { rechercher(yytext,"Separateur","","",3); return diaz;}
"{"  { rechercher(yytext,"Separateur","","",3); return acco;}
"}"  { rechercher(yytext,"Separateur","","",3); return accf;}
":"  { rechercher(yytext,"Separateur","","",3);  return dpt;}
"="  { rechercher(yytext,"Separateur","","",3);  return eg; }
":="  { rechercher(yytext,"Separateur","","",3); return aff;}
"["  { rechercher(yytext,"Separateur","","",3); return crov;}
"]"  { rechercher(yytext,"Separateur","","",3); return crfm;}
"$"  { rechercher(yytext,"Separateur","","",3); return fin;}
"//"  { rechercher(yytext,"Separateur","","",3); return sep;}
[ \t] {}
\n   {nb_ligne++;}
. printf ("Entité non reconnue a ligne %d \n", nb_ligne);
%%
```

syn.y

```
%{
 extern int nb_ligne;
%}
%union {
   int entier;
   char* str;
}

%token mc_import mc_stdio mc_array mc_pgm mc_entier mc_var mc_const divis
  diaz acco accf dpt eg aff crov crfm fin sep cst
%start S
%%
S: LIST_BIB mc_pgm idf acco Program accf { printf(" Le programme est syntaxiquement correcte.\n"); YYACCEPT; }
;
Program: LISTE_DEC LISTE_INSTR
;
LISTE_DEC: DEC LISTE_DEC
  |
;
DEC: DEC_VAR
 |DEC_CONST
;
DEC_VAR: mc_var TYPE dpt LIST_IDF fin
  | mc_var TYPE dpt DEC_TAB fin
;
DEC_TAB: idf crov cst crfm
;
DEC_CONST: mc_const TYPE dpt LIST_IDF_CST fin
;
LIST_IDF_CST : idf eg cst sep LIST_IDF_CST
    | idf eg cst
;
LIST_IDF: idf sep LIST_IDF
  | idf
;
TYPE: mc_entier
;
LISTE_INSTR: INST_AFF LISTE_INSTR
    |
;
INST_AFF: idf aff EXPRESSION fin
;
EXPRESSION: idf
   | cst
   | idf divis cst
;
LIST_BIB: BIB LIST_BIB
  |
;
BIB: diaz mc_import NOM_BIB
;
NOM_BIB: mc_array
  |mc_stdio
;
%%
main ()
{
 initialization();
 yyparse();
 afficher();
 }
yywrap ()
 {}
int yyerror ( char* msg )
{
 printf ("Erreur Syntaxique a ligne %d a colonne %d \n", nb_ligne,Col);
}
```

Résultat de l'execution:

TS-Exmpl2 > test.txt

You, 16 hours ago | 1 author (You)

```
#import Array
#import Stdio
Programme ACAD_B
{
   VAR  INTEGER : A // B // X $
   VAR  INTEGER : Tab[5] $
   CONST  INTEGER : C=5// R=15 $
   A:=5$
   B:=5$
   A:=A/0$
   B:=C$
}
```

You, 16 hours ago • Table d

```
/**************Table des symboles IDF*************/
-------------------------------------------------------------------
        | Nom_Entite |  Code_Entite | Type_Entite | Val_Entite
-------------------------------------------------------------------
        |     ACAD_B |          IDF |             |
        |          A |          IDF |             |
        |          B |          IDF |             |
        |          X |          IDF |             |
        |        Tab |          IDF |             |
        |          C |          IDF |             |
        |          R |          IDF |             |
```

```
/**************Table des symboles mots cles*************/
-------------------------------------
        | NomEntite |  CodeEntite |
-------------------------------------
        |    import |     Mot cle |
        |     Array |     Mot cle |
        |     Stdio |     Mot cle |
        | Programme |     Mot cle |
        |       VAR |     Mot cle |
        |   INTEGER |     Mot cle |
        |     CONST |     Mot cle |
```

Résultat de l'execution:

TS-Exmpl2 > test.txt

You, 16 hours ago | 1 author (You)

```
#import Array
#import Stdio
Programme ACAD_B
{
   VAR  INTEGER : A // B // X $
   VAR  INTEGER : Tab[5] $
   CONST  INTEGER : C=5// R=15 $
   A:=5$
   B:=5$
   A:=A/0$
   B:=C$
}
```

You, 16 hours ago • Table

```
/***************Table des symboles separateurs*************/
----------------------------------------
        | NomEntite |  CodeEntite |
----------------------------------------
        |         # |  Separateur |
        |         { |  Separateur |
        |         : |  Separateur |
        |        // |  Separateur |
        |         $ |  Separateur |
        |         [ |  Separateur |
        |         ] |  Separateur |
        |         = |  Separateur |
        |        := |  Separateur |
        |         / |  Separateur |
        |         } |  Separateur |
```

TABLE DE SYMBOLES

2ème version

Programme source

Program L3Compil
integer x;
Const reel y=5;
begin
x=y+1;
end

NomEntité	CodeEntité	Type	Const
L3Compil	Idf	/	Non
x	Idf	integer	Non
y	Idf	reel	Oui
.....			

Déclaration de la table de symbole

Lexical.l

```
%{
//structure de la table de symbole
typedef struct
{
char NomEntite[20];
char CodeEntite[20];
} TypeTS;
//Initialisation d'un tableau pour stocker les éléments de la table des symboles.
TypeTS ts[100];
// compteur global pour la table de symbole
int CpTS = 0;
```

Fonction de recherche dans la TS

- la fonction **recherche**: cherche est ce que l'entité existe ou non dans la table de symbole.
- Entrée: Entité
- Sortie:

Si l'entité existe déjà, il va renvoyer sa position.

Sinon -1, s'il elle n'existe pas dans la TS.

```
int recherche(char entite[])
{
 int i = 0;
 while (i < CpTS)
 {
  if (strcmp(entite, ts[i].NomEntite) == 0)
   return i;
  i++;
 }

 return -1;
}
```

Fonction d'insertion dans la TS

- fonction qui va insérer les entités de programme dans la table de symbole

```
void inserer(char entite[], char code[])
{
 if (recherche(entite) == -1) //elle n'existe pas
 {
  strcpy(ts[CpTS].NomEntite, entite);
  strcpy(ts[CpTS].CodeEntite, code);
  CpTS++;
 }
}
```

Fonction d'affichage de la TS

- Pour afficher le contenu de la table de symboles.

```
void afficher()
{
 printf("\n/**************Table des symboles *****************/\n");
 printf("_____________________________________\n");
 printf("\t| NomEntite | CodeEntite | \n");
 printf("_____________________________________\n");
 int i = 0;
 while (i < CpTS)
 {
  printf("\t|%10s |%12s |\n", ts[i].NomEntite, ts[i].CodeEntite);
  i++;
 }
}
```

Utilisation du fichier Fonctions.h

- La déclaration de la table des symboles (TS) et les fonctions associées peuvent encombrer le fichier lexical.l.
- Pour plus de clarté, il est conseillé de les déplacer dans un fichier d'en-tête, par exemple **Fonctions.h**, et de l'inclure dans lexical.l avec:

#include "Fonctions.h"

Fonctions.h

```
//Structure de la TS
#include <stdio.h>
#include <stdlib.h>
#include<string.h>
typedef struct
{
 char NomEntite[20];
 char CodeEntite[20];
} TypeTS;

TypeTS ts[100];

// init compteur
int CpTS = 0;

void inserer(char entite[], char code[])
{
}

void recherche (char entite[])
{
}

void afficher()
{
}
```

Exemple:

Voici le programme suivant:

```
Program  L3ISILA
PDec
integer x,i,j;
Reel  y,z,a;
InBEGIN
 i=2;
 j=i/0;
 x=x+1;
InEnD
```

Lexical.l

```
%{
#include "syn.tab.h"
#include "Fonctions.h"
int nb_ligne=1;
extern YYSTYPE yylval;
extern int col;
%}
lettre [a-zA-Z]
chiffre [0-9]
idf {lettre}({lettre}|{chiffre})*
cst [1-9][0-9]*|0
%%
Program {return mc_program;}
PDec {return mc_pdec;}
integer {yylval.str=strdup(yytext); return mc_integer;}
Reel  {yylval.str=strdup(yytext); return mc_real;}
InBEGIN {return mc_inbegin;}
InEnD {return mc_inend;}
[,;=/+] {return yytext[0];}
{cst} {yylval.entier=atoi(yytext); return cst;}
{idf} {yylval.str=strdup(yytext); inserer(yytext,"idf"); return idf;}
[ \t]
\n {nb_ligne++;col=1;}
. {printf ("erreur lexical sur l'entite %s a la ligne %d a la colonne %d",yytext,nb_ligne,col);
return err;}
%%
```

syn.y

```
%{
 extern int nb_ligne;
extern int col;
 void yyerror(char *msg);
%}
%start S
%union
{
int entier;
char* str;
}
%token mc_inbegin mc_inend mc_program mc_pdec err
%token <str>idf <str>  mc_integer <str> mc_real
%token <entier> cst
%%
S: Header mc_pdec DECLARATION INSTRUCTION
{ printf ("programme syntaxiquement correcte . \n"); YYACCEPT;}
;
Header: mc_program idf
;
DECLARATION: type listeparams ';' DECLARATION
    | type listeparams ';'
;
listeparams: listeparams ',' idf
 | idf
 ;
type: mc_integer
  | mc_real
;
INSTRUCTION: mc_inbegin ListeInstr mc_inend
;
ListeInstr: ListeInstr Instruction
   | Instruction
;
Instruction : instaff
    | instdiv
    | instadd
;
instaff: idf '=' idf ';'
  | idf '=' cst ';'
;
instdiv: idf '=' idf '/' idf ';'
  | idf '=' idf '/' cst ';'
  ;
instadd: idf '=' idf '+' idf ';'
 | idf '=' idf '+' cst ';'
 ;
%%
void yyerror(char *msg) {
 printf("Erreur syntaxique %s, a la ligne %d \n", msg, nb_ligne);
}
main ()
{
yyparse();
afficher();
}
yywrap(){
}
```

Résultat de l'execution:

TSv1-ISIL A > test.txt

```
Program  L3ISILA
PDec
integer x,i,j;
Reel  y,z,a;
InBEGIN
 i=2;
 j=i/0;
 x=x+1;
InEnD
```

```
programme syntaxiquement correcte .

/***************Table des symboles ******
_____________________________________
        | NomEntite |  CodeEntite  |
_____________________________________
        |   L3ISILA |          idf |
        |         x |          idf |
        |         i |          idf |
        |         j |          idf |
        |         y |          idf |
        |         z |          idf |
        |         a |          idf |
```